

Lista: Introdução ao Java.

Nome: _____ **Matrícula:** _____

Exercícios baseados no capítulo 13 do livro Modern Programming Languages.

Instruções: A menos que o exercício indique o contrário, você deve implementar suas soluções em estilos imperativos e orientados a objetos. Teste suas soluções com uma classe `Main`.

Exercício 01: Adicione um método de instância `contains` à classe `IntList`, para que `x.contains(n)` retorne verdadeiro se o valor `int n` ocorrer em `IntList x` e retorne falso caso contrário.

Exercício 02: Adicione um método de instância `equals` à classe `IntList`, para que `x.equals(y)` retorne verdadeiro se `IntList x` e `IntList y` tiverem exatamente os mesmos inteiros na mesma ordem, e retorne falso caso contrário. Deve ser verdade que `x.equals(y)` seja sempre equivalente a `y.equals(x)`. Também deve ser verdade que se `x == y` então `x.equals(y)`, embora o inverso não seja necessariamente verdadeiro.

Exercício 03: Adicione um método de instância `append` à classe `IntList`, para que `x.append(y)` retorne uma `IntList` que é igual à `IntList x` seguida pela `IntList y`. Não deve haver efeito colateral em `x` ou `y`. (Dica: Você precisará fazer uma cópia de `x`.)

Exercício 04: Adicione um método de instância `reverse` à classe `IntList`, para que `x.reverse()` retorne uma `IntList` que é o reverso da `IntList x`. Não deve haver efeito colateral em `x`.

Exercício 05: Adicione um método de instância `reverseMe` à classe `IntList`, para que `x.reverseMe()` não retorne valor, mas tenha o efeito colateral de reverter o conteúdo de `x`. (Isso é mais fácil se você começar fazendo uma cópia invertida de `x`.)

Exercício 06: Adicione um método de instância `sort` à classe `IntList`, para que `x.sort()` retorne uma `IntList` que é uma versão de `IntList x`, ordenada

em ordem não decrescente. Você pode usar qualquer algoritmo de ordenação que preferir. Não deve haver efeito colateral em `x`.

Exercício 07: Crie uma classe `Int` com os seguintes membros:

- a. Um campo para armazenar um valor `int`.
- b. Um construtor para que `new Int(x)` crie um objeto `Int` que armazene o valor `int x`.
- c. Um método de instância `toString` para que `x.toString()` retorne o valor do objeto `Int x` em formato de `String`.
- d. Um método de instância `plus` para que `x.plus(y)` retorne um novo objeto `Int` cujo valor é o valor de `Int x` mais o valor de `Int y`. Não deve haver efeitos colaterais.
- e. Métodos de instância `minus`, `times` e `div`, semelhantes ao método `plus` descrito acima. (O método `div` deve realizar divisão inteira, como o operador `/` em valores `int`.)
- f. Um método de instância `isPrime` para que `x.isPrime()` retorne verdadeiro se o valor de `Int x` for um número primo.